We don't need to design the entire API up front. Let's assume we'll be POSTing some key-value pairs to the /expenses endpoint. As many web APIs do, we'll support sending and receiving data in JSON format.[7] Because JSON objects convert to Ruby hashes with string keys, our example data will also have string keys. For example, we'll say { 'payee' => 'Starbucks' } instead of { payee: 'Starbucks' }.

A Quick Look at HTTP APIs

Our expense tracker API is built on Hypertext Transfer Protocol (HTTP). This is the same protocol that your web browser uses to connect to websites, but in our case the requests won't necessarily be coming from a browser. A command-line program, a desktop GUI, or a set of RSpec examples may generate these requests.

We're only going to use two of the most basic features of HTTP in these examples:

- A GET request reads data from the app.
- A POST request modifies data.

There's a lot more to HTTP than just this. For more information, see one of the many tutorials available.[a]

a. https://code.tutsplus.com/tutorials/a-beginners-guide-to-http-and-rest--net-16340

To get the data into and out of our app, we'll use several different helper methods from Rack::Test::Methods. As you can see, you can include Ruby modules into an RSpec context, just like you're used to doing inside Ruby classes.

The first Rack::Test helper we'll use is post. This will simulate an HTTP POST request, but will do so by calling our app directly rather than generating and parsing HTTP packets.

We don't have an app yet, but let's go ahead and run our specs anyway. This will give us a hint on what to implement next:

```
$ bundle exec rspec
F

Failures:

  1) Expense Tracker API records submitted expenses
     Failure/Error: post '/expenses', JSON.generate(coffee)

     NameError:
       undefined local variable or method `app' for                    ↩
       #<RSpec::ExampleGroups::ExpenseTrackerAPI:0x007fef0404f560>

« truncated »
```

7. http://json.org